

Đề Cương chi tiết học phần (Kế hoạch giảng dạy)

1.Tên học phần: Toán kỹ thuật điện tử - máy tính Mã học phần: ECEM244164

2.Tên Tiếng Anh: Electronic and Computer Engineering Mathematics

3.Số tín chỉ: 4 tín chỉ (4/0/0) (4 tín chỉ lý thuyết, 0 tín chỉ thực hành/thí nghiệm)

Bài giảng về Vibe Coding tạo code để ứng dụng Biến đổi Laplace vào phân tích mạch điện,

Sưu tầm và Biên soạn bổ sung phần Vibe Coding
With the support of Gemini

Prof Happy AI

Date: May 9, 2026

Theo syllabus của học phần:

Chương 3: Biến đổi Laplace và ứng dụng Nội dung GD trên lớp: (4) 3.1 Phép biến đổi Laplace 2.2 Phép biến đổi Laplace của đạo hàm và tích phân 2.3 Biến đổi Laplace của hàm bước đơn vị 2.3 Ứng dụng của phép biến đổi Laplace giải bài toán mạch điện

Bài giảng về Vibe Coding tạo code để ứng dụng Biến đổi Laplace vào phân tích mạch điện,

Dưới đây là bản dịch tiếng Việt cho đoạn nội dung trên:

Biến đổi Laplace và Phương trình Vi phân Mạch điện

Trong lĩnh vực mạch điện, nhiều hệ thống được điều khiển bởi các phương trình vi phân tuyến tính do đặc tính của các linh kiện như điện trở, cuộn cảm và tụ điện. Việc

áp dụng phép biến đổi Laplace vào các phương trình này sẽ chuyển đổi các toán tử vi phân thành các đa thức của biến s .

Xét một mạch bậc một đơn giản:

$$L \frac{di(t)}{dt} + Ri(t) = v(t) \quad (13.2)$$

Áp dụng phép biến đổi Laplace và giả sử các điều kiện đầu bằng không:

$$LsI(s) + RI(s) = V(s) \quad (13.3)$$

Phép toán này chuyển đổi bài toán thành một phương trình đại số theo biến $I(s)$:

$$I(s) = \frac{V(s)}{Ls + R}$$

Dưới đây là bản dịch tiếng Việt cho đoạn nội dung trên:

✓ Ứng dụng trong Phân tích Mạch điện

Trong phân tích mạch thực tế, phép biến đổi Laplace được sử dụng để đơn giản hóa các phương trình hệ thống được thiết lập từ các định luật mạch điện. Nó cho phép xử lý các điều kiện đầu một cách trực tiếp và cung cấp một khung làm việc để áp dụng đại số sơ đồ khối. Một ví dụ được đưa ra dưới đây:

Xét một mạch RLC nối tiếp được mô tả bởi phương trình:

$$v(t) = Ri(t) + L \frac{di(t)}{dt} + \frac{1}{C} \int_0^t i(\tau) d\tau \quad (13.10)$$

Áp dụng phép biến đổi Laplace ta được:

$$V(s) = RI(s) + LsI(s) + \frac{I(s)}{Cs} \quad (13.11)$$

Giải phương trình để tìm dòng điện $I(s)$:

$$I(s) = \frac{V(s)}{R + Ls + \frac{1}{Cs}} \quad (13.12)$$

Ví dụ này minh họa cách phép biến đổi Laplace chuyển đổi các phương trình vi phân phức tạp thành các dạng đại số đơn giản hơn, giúp tạo điều kiện thuận lợi cho việc tìm nghiệm phân tích và nghiệm số trong thiết kế cũng như phân tích mạch điện.

Dưới đây là đoạn mã Python cung cấp các phép tính cốt lõi liên quan đến việc triển khai các phương pháp biến đổi Laplace trong phân tích mạch điện, bao gồm việc biến đổi các phương trình vi phân và giải tìm đáp ứng của hệ thống.

Dưới đây là đoạn mã Python (Vibe Coding) cung cấp các phép tính cốt lõi liên quan đến việc triển khai các phương pháp biến đổi Laplace trong phân tích mạch điện, bao gồm việc biến đổi các phương trình vi phân và giải tìm đáp ứng của hệ thống.

```
## Chương trình do AI tạo ra dùng Vibe Coding
import numpy as np
from scipy.integrate import quad
from scipy.signal import lti, impulse, invres

def laplace_transform(f, s, t0=0):
    """
    Tính toán biến đổi Laplace của một hàm f(t).
    :param f: Hàm số thời gian.
    :param s: Biến số phức (thực chất tích phân này hỗ trợ s thực trong hàm
    """
    result, _ = quad(lambda t: f(t) * np.exp(-s * t), t0, np.inf)
    return result

def inverse_laplace_transform(poles, residues, t_max, num_points=500):
    """
    Tính toán biến đổi Laplace ngược.
    Sử dụng invres để chuyển từ thặng dư/cực sang đa thức Tử số/Mẫu số.
    """
    # Chuyển đổi khai triển phân thức từng phần về dạng đa thức hệ thống
    num, den = invres(residues, poles, [0])

    # Khởi tạo hệ thống Tuyến tính Dừng (LTI)
    system = lti(num, den)

    # Tạo miền thời gian và tính đáp ứng xung (impulse response)
    t_space = np.linspace(0, t_max, num_points)
    t, y = impulse(system, T=t_space)
    return t, y

def solve_first_order_system(L, R, V_s):
    '''Giải mạch bậc một:  $I(s) = V_s / (L*s + R)$ '''
    return lambda s: V_s / (L * s + R)

def analyze_rlc_circuit(V_s, R, L, C):
    '''Giải mạch RLC:  $I(s) = V_s / (R + L*s + 1/(C*s))$ '''
    return lambda s: V_s / (R + L * s + 1 / (C * s))

# --- Chương trình chính ---
if __name__ == "__main__":
    # 1. Các thông số giả định cho bài toán phân tích mạch
    # Giả sử ta có hàm truyền  $I(s) = 0.5/(s + 0.2) + 0.5/(s + 0.5)$ 
```

```

poles = [-0.2, -0.5]
residues = [0.5, 0.5]
t_max = 10

# 2. Tính toán biến đổi Laplace ngược để lấy đáp ứng thời gian
time, current_response = inverse_laplace_transform(poles, residues, t_max)

# 3. Hiển thị kết quả
print("--- Kết quả phân tích mạch điện ---")
print("Đáp ứng dòng điện tại 5 thời điểm đầu tiên:")
# np.round giúp hiển thị số liệu gọn gàng
print(np.round(current_response[:5], 6))

print(f"\nGiá trị dòng điện tại t=0 (Lý thuyết là 1.0): {current_response[0]}")

# 4. Minh họa tính toán miền Laplace (tích phân số)
# Thử nghiệm với hàm f(t) = e^(-0.2t)
f_t = lambda t: np.exp(-0.2 * t)
s_val = 1.0
F_s = laplace_transform(f_t, s_val)
print(f"\nKiểm tra Laplace Transform của e^(-0.2t) tại s={s_val}: {F_s:.4f}")

```

```

--- Kết quả phân tích mạch điện ---
Đáp ứng dòng điện tại 5 thời điểm đầu tiên:
[1.          0.993015  0.986088  0.979218  0.972404]

Giá trị dòng điện tại t=0 (Lý thuyết là 1.0): 1.00

Kiểm tra Laplace Transform của e^(-0.2t) tại s=1.0: 0.8333

```

Trực quan hóa: việc trực quan hóa bằng đồ thị sẽ giúp chúng ta thấy rõ sự suy giảm của dòng điện theo thời gian, giúp sinh viên dễ dàng hình dung hơn về đáp ứng của mạch điện.

Dưới đây là mã nguồn đã tích hợp thêm thư viện Matplotlib để vẽ đồ thị:

Trực quan hóa: việc trực quan hóa bằng đồ thị sẽ giúp chúng ta thấy rõ sự suy giảm của dòng điện theo thời gian, giúp sinh viên dễ dàng hình dung hơn về đáp ứng của mạch điện. Dưới đây là mã nguồn đã tích hợp thêm thư viện Matplotlib để vẽ đồ thị:

```

import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import lti, impulse, invres

def inverse_laplace_transform(poles, residues, t_max, num_points=500):

```

```

'''
Tính toán biến đổi Laplace ngược và trả về dữ liệu thời gian/đáp ứng.
'''

# Chuyển đổi từ residues/poles sang đa thức hệ thống
num, den = invres(residues, poles, [0])
system = lti(num, den)

t_space = np.linspace(0, t_max, num_points)
t, y = impulse(system, T=t_space)
return t, y

def visualize_response(t, y, title="Đáp ứng của mạch điện theo thời gian"):
    '''
    Trực quan hóa đáp ứng bằng Matplotlib.
    '''

    plt.figure(figsize=(10, 6))
    plt.plot(t, y, label='i(t)', color='blue', linewidth=2)

    # Thêm các đường lưới và nhãn
    plt.title(title, fontsize=14)
    plt.xlabel('Thời gian (giây)', fontsize=12)
    plt.ylabel('Dòng điện i(t) (Ampe)', fontsize=12)
    plt.grid(True, which='both', linestyle='--', alpha=0.7)
    plt.axhline(y=0, color='black', linewidth=1)
    plt.axvline(x=0, color='black', linewidth=1)
    plt.legend()

    # Đánh dấu giá trị tại t=0
    plt.scatter([t[0]], [y[0]], color='red')
    plt.annotate(f'i(0) = {y[0]:.2f}', (t[0], y[0]), textcoords="offset poin

    plt.show()

# --- Thực thi chương trình ---
if __name__ == "__main__":
    # Giả sử đáp ứng  $i(t) = 0.5e^{(-0.2t)} + 0.5e^{(-0.5t)}$ 
    poles = [-0.2, -0.5]
    residues = [0.5, 0.5]
    t_max = 15 # Tăng thời gian để thấy rõ sự suy giảm về 0

    # 1. Tính toán
    t, y = inverse_laplace_transform(poles, residues, t_max)

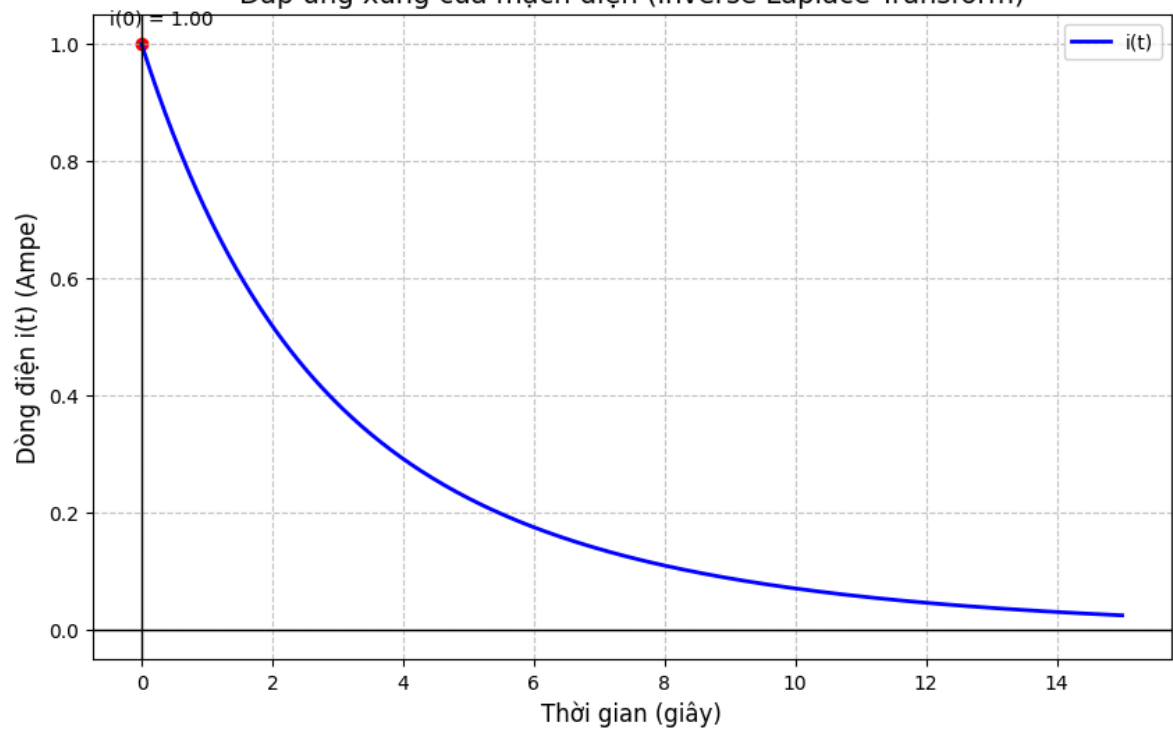
    # 2. In kết quả số học
    print(f"Giá trị tại t=0: {y[0]:.2f}")

    # 3. Vẽ đồ thị
    visualize_response(t, y, title="Đáp ứng xung của mạch điện (Inverse Lapl

```

Giá trị tại $t=0$: 1.00

Đáp ứng xung của mạch điện (Inverse Laplace Transform)



Start coding or [generate](#) with AI.